

Mini Game Hub

CS-108 Project

Yogi & Praneeth

April 2026

Abstract

This report is about the development of the Mini Game Hub, a secure, multi-user platform that combines Bash scripting for user authentication and Python (Pygame) for gameplay. Two authenticated players can select a game from the menu, play through a graphical interface, and save their results to a persistent leaderboard.

Contents

1	Introduction	2
2	Libraries and Tools	2
3	System Architecture	2
3.1	Directory Structure	2
4	Features	2
5	Core Implementation Logic	3
5.1	Authentication System (Bash)	3
5.2	Generic Game Class (Inheritance)	3
5.3	Game-Specific Logic	3
6	Leaderboard and Analytics	6
7	Hurdles and Solutions	6
8	The "10-Hour" Plan	6

1 Introduction

The Mini Game Hub is a pygame,bash platform where two players can log in and compete in various board games. The system's architecture focuses on two main areas: security (using SHA-256 hashing for passwords) and efficient game logic using NumPy matrix operations.

2 Libraries and Tools

- **Bash / Shell:** Used for the authentication of the system and formatting the terminal-based leaderboard.
- **Pygame-CE:** Used to build the GUI and handle real-time user input.
- **NumPy:** Used for board representations and win-condition slicing to keep the game logic fast and efficient.
- **Matplotlib:** Used to generate post-game analytics, including bar and pie charts.

3 System Architecture

3.1 Directory Structure

```
cs108-project/  
├── main.sh ... (Entry point & Auth)  
├── game.py ... (Main Menu & Selection)  
├── leaderboard.sh ... (Bash Statistics)  
├── matplot.py ... (Analytics)  
├── users.tsv ... (Hashed Credentials)  
├── history.csv ... (Game Records)  
├── games/  
│   ├── base_game.py ... (Parent Class)  
│   ├── tictactoe.py  
│   ├── othello.py  
│   └── connect4.py  
├── report.tex  
└── Makefile
```

4 Features

- **User Authentication:** A secure login system using SHA-256 hashing to protect the user passwords.
- **Multi-Game Support:** Includes full versions of Tic-Tac-Toe, Othello, and Connect 4.
- **Turn-Based Game play:** A reliable system for managing player turns and game flow.
- **Win Detection:** Optimized algorithms that instantly detect winning conditions.
- **Leaderboard System:** Tracks player wins and losses over time.
- **Data Persistence:** All match records are saved in CSV format.
- **Data Visualization:** Automatically generates and opens the performance graphs using Matplotlib.
- **Menu Interface:** A user-friendly GUI for easy navigation and game selection.

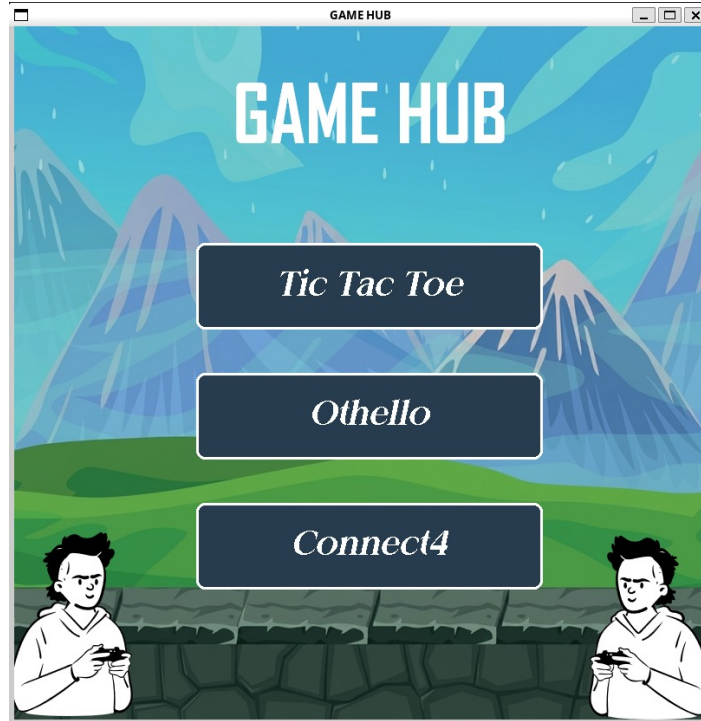


Figure 1: The Mini Game Hub Main Menu Interface

5 Core Implementation Logic

5.1 Authentication System (Bash)

The `main.sh` script acts as the entry point. It uses the `sha256sum` tool to hash passwords before storing them in `users.tsv`. This ensures that even if the data file is accessed, the actual passwords remain secure.

5.2 Generic Game Class (Inheritance)

We created a `Base` class to handle shared properties like player names, turn switching. Specific games like `Tictactoe` and others too inherit from this class.

5.3 Game-Specific Logic

Tic-Tac-Toe (10×10, Five-in-a-Row)

This version of Tic-Tac-Toe is played on a 10×10 grid, where the objective is to place five consecutive marks in any direction (horizontal, vertical, or diagonal). Internally, the board state is stored using a NumPy array called `used`, where each cell contains `None` (empty), 1 (Player X), or 0 (Player O).

When a player makes a move, the corresponding cell in the array is updated with the player's value, and the turn is switched. Instead of using nested loops for checking wins, the implementation relies on efficient NumPy operations.

To detect a win, a boolean matrix is first created:

$$x = (\text{used} == 1 - \text{player})$$

This isolates all positions belonging to the last player who made a move.

For horizontal checks, the board is sliced into overlapping windows of length 5 (e.g., columns 0–4, 1–5, ..., 5–9). The `all(axis=1)` function verifies whether all elements in each slice are True. If any row satisfies this condition, a win is declared.

A similar approach is used for vertical checks, but slicing is applied across rows instead of columns.

For diagonal detection, the logic becomes more interesting. Instead of checking each diagonal individually, multiple shifted subarrays (like 6×6 blocks) are extracted and combined using element-wise logical AND operations. If the same position across all these arrays is True, it indicates five consecutive marks diagonally. This method avoids explicit iteration and makes the check extremely fast.

The same idea is applied for both $\backslash$ and $/$ diagonals by appropriately slicing the matrix. If any condition is satisfied, the game is marked as over immediately.

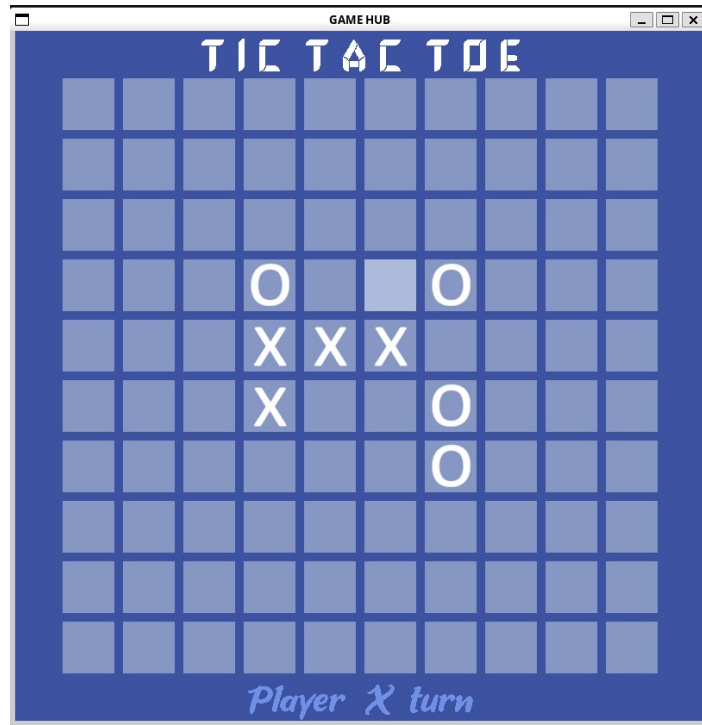


Figure 2: Tic-Tac-Toe

Connect 4

Connect 4 is played on a 7×7 grid where the goal is to align four consecutive discs. The board is again stored using a NumPy array, and each column has an associated `level` value that tracks how many discs are already placed in that column.

When a player selects a column, the disc is assigned to the lowest available position using:

$$\text{row} = 6 - \text{level}[\text{col}]$$

This simulates gravity, ensuring that pieces stack correctly from bottom to top.

After placing a disc, the turn switches to the other player, and the win-checking process begins.

The win detection logic closely mirrors the Tic-Tac-Toe approach but uses sequences of length 4 instead of 5. Horizontal checks are performed by slicing the board into overlapping groups of four columns and applying `all(axis=1)`. Vertical checks use similar slicing across rows.

For diagonal detection, multiple 4×4 submatrices are extracted from the board. These submatrices are shifted step-by-step and combined using logical AND operations. If any position remains True after combining, it indicates four aligned discs diagonally.

This method ensures that all possible diagonal combinations are checked efficiently without explicitly looping over each diagonal. As soon as any valid sequence is found, the game state is updated to indicate completion.

Othello (Reversi)

Othello is played on an 8×8 grid, where the main challenge lies not in placement, but in flipping the opponent's discs. The board is represented using a NumPy array where 0 denotes Black, 1 denotes White, and `None` indicates an empty cell.

The core logic revolves around validating moves and flipping discs correctly. A move is only valid if placing a disc captures at least one opponent disc. This is handled by the `check_and_flip` function, which checks all four directions: horizontal, vertical, and two diagonals.

At the heart of this process is the `checkline` function. Given a one-dimensional slice of the board (row, column, or diagonal), it examines both forward and backward directions from the chosen position. It searches for a sequence where opponent discs are followed by a disc of the current player. If such a pattern is found, all intermediate discs are flipped.

For horizontal and vertical checks, rows and columns are directly extracted from the array. For diagonal checks, NumPy's `diagonal()` function is used. In the case of the anti-diagonal, the board is flipped using `flipplr()` before extracting the diagonal.

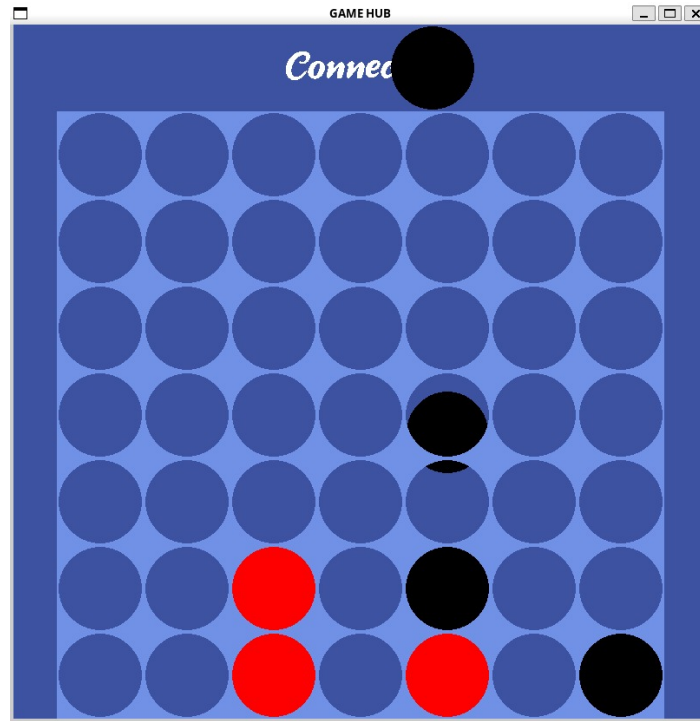


Figure 3: Connect4

If any direction results in a successful flip, the move is considered valid, and the board is updated in-place.

The game also includes a recursive function `has_any_valid_move`, which scans all empty cells to determine whether a player has at least one valid move. If a player cannot move, the turn is passed back. If neither player has a valid move, the game ends.

Finally, the winner is determined by counting the number of discs using:

```
np.sum(used == 0) and np.sum(used == 1)
```

This implementation focuses heavily on array manipulation and avoids unnecessary loops, making the logic both efficient and clean.

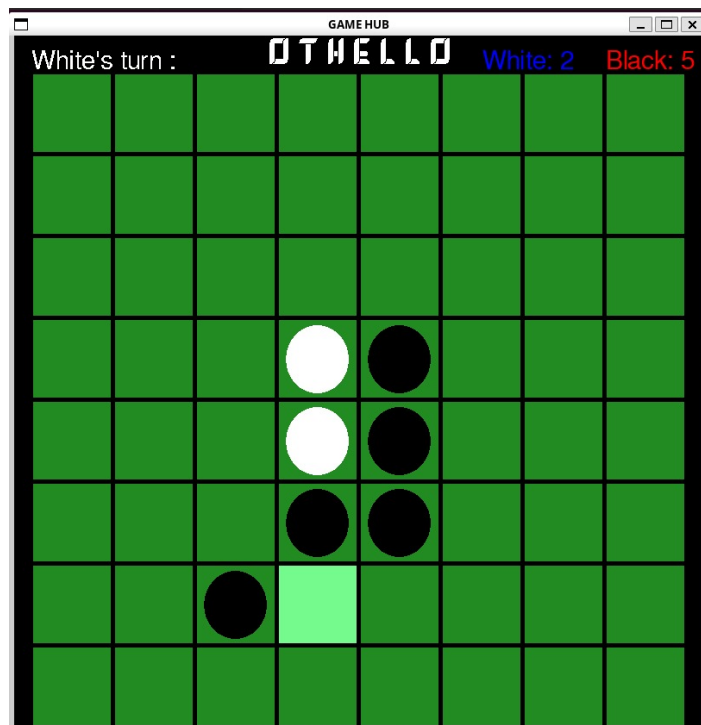


Figure 4: Othello

6 Leaderboard and Analytics

After each match, the Python engine appends the result to `history.csv`. The `leaderboard.sh` script then processes this data using `awk` and `sort` to rank players. At the same time, `matplotlib.py` creates visual charts of the players' performance history.

7 Hurdles and Solutions

- **Hurdle:** Passing data smoothly between Bash scripts and Python scripts.
Solution: Used `subprocess` and shared CSV files to communicate between languages.
- **Hurdle:** Checking win conditions on large grids without slow loops.
Solution: Leveraged NumPy functions like `diagonal()`, `all()`, and `flipplr()` for instant matrix checks.

8 The "10-Hour" Plan

If we had 10 additional hours, we would implement a Single Player mode featuring an machine opponent so users can play even when a second player is not available. definitely we would improve our project .

References

- [1] Python inheritance: <https://docs.python.org/3/tutorial/classes.html>
- [2] Pygame CE Documentation: <https://inventwithpython.com/pygame/>

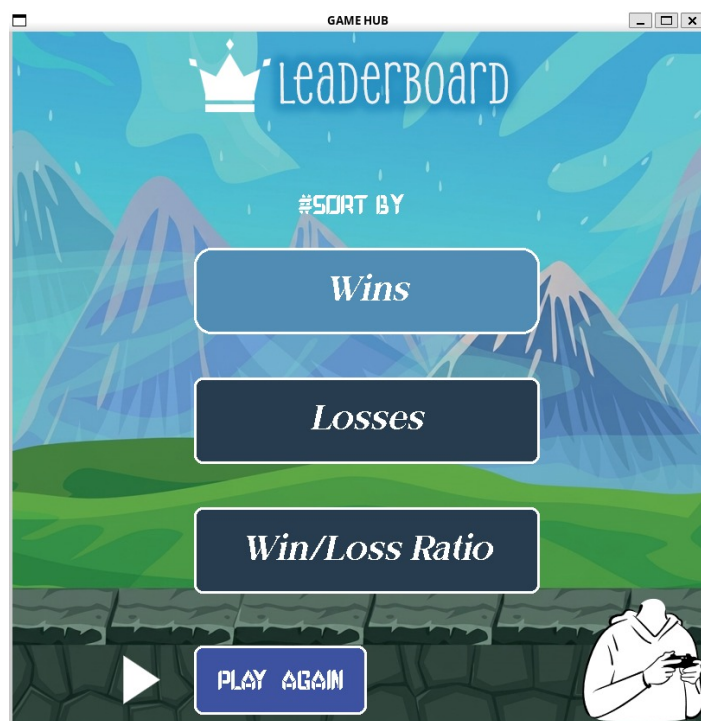


Figure 5: Post-game Leaderboard and Analytics Dashboard